Assignment 5 Strategy Pattern

Identification

Assignment title: Strategy Pattern

Full name: Yermekkyzy Assel

Group: SE-2524

Topic: Option B — Calculator

GitHub repository link: https://github.com/yerassell/assignment5-strategy-pattern.git

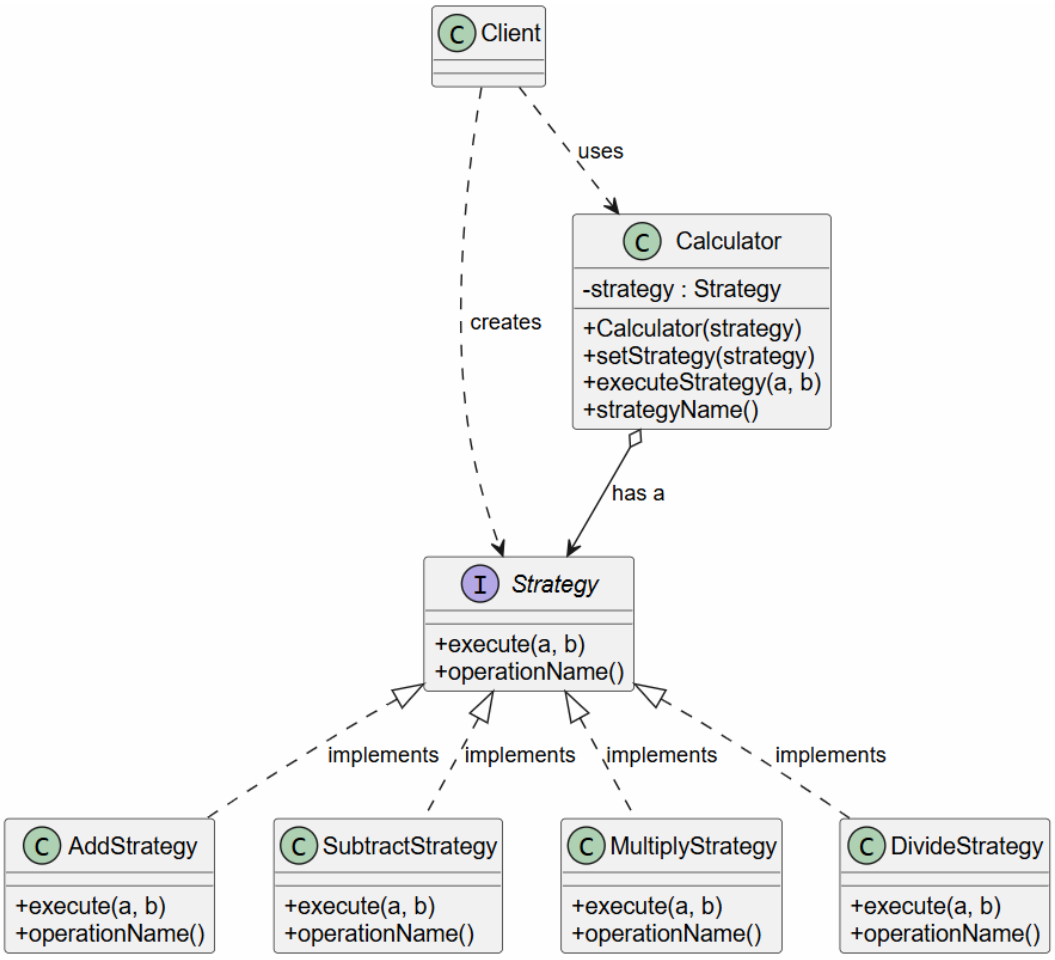
Last commit: 4d58c6e0988cba46f1fa3fdb501e4597f6372b29

Introduction

This is an implementation of Strategy design pattern in Calculator, the topic that I chose for this assignment. The chosen domain is Calculator, the algorithm family is addition, subtraction, multiplication and division.

The strategy pattern solves the problem of using conditional logic. Instead of using if or switch statements to check the operation type, the Strategy pattern creates classes for each algorithm family. The context class Calculator only calls the execute method without knowing which operation it is doing.

UML class diagram



Picture 1. The UML class diagram

Clean Code

1. *Elimination of conditional/branching logic in favor of polymorphism*
   Explanation:
   Instead of using if or switch statements to choose the operation, each operation has its own class. The context class Calculator calls execute method without knowing the operation type
   Excerpt (Calculator.java):

```
public double executeStrategy(double a, double b) {
    return strategy.execute(a, b);
}
```

2. *Meaningful, intention-revealing names for each strategy*
   Explanation:
   The names are clear and self-explanatory. Users understand what each class name or method name mean and does by just reading it
   Excerpt (Calculator.java):

```
public class Calculator {
    private Strategy strategy;

    public Calculator (Strategy strategy) {…}
    public void setStrategy(Strategy strategy) {…}
    public double executeStrategy(double a, double b) {…}
    public String strategyName(){…}
```

3. *Single Responsibility: each strategy does exactly one algorithm, nothing else*
   Explanation:
   Each class has one job. AddStrategy does addition, SubtractStrategy does subtraction and so on
   Excerpt (AddStrategy.java):

```
public class AddStrategy implements Strategy {
    public double execute(double a, double b) {
        return a + b;
    }
    public String operationName(){
        return "Addition";
    }
}
```

4. *Context remains algorithm-agnostic (no knowledge of concrete strategy internals)*
   Explanation:
   The context class Calculator does not know about the concrete strategies. It only depends on the interface Strategy, so it imports only Strategy. Any added or deleted concrete strategy does not affect Calculator class.
   Excerpt (Calculator.java):

```
import strategy.Strategy;

public class Calculator {
    private Strategy strategy;
```

5. *Easy extensibility demonstrated (adding a new strategy without touching existing classes)*
   Explanation:

Adding a new class will not change the existing code. If PowerStrategy class was added to execute power of a number, then the Strategy, Calculator or any other concrete strategies will not change. There would be a new line in the Client class which will set strategy on runtime.
Excerpt (Client.java if a new PowerStrategy class was added):

```
calculator.setStrategy(new PowerStrategy());
System.out.println(calculator.strategyName() + ": " +
calculator.executeStrategy(6, 2));
```

Strategy vs. State

Strategy and State pattern look similar but they solve different problems. They both have the interface, the context class that delegates to it.

The Strategy pattern focuses on algorithm interchangeability. The client chooses a different algorithm and the Context changes its behavior. The strategies do not know about each other and do not trigger transitions. So here the Client makes the choice.

The State pattern can be viewed as Strategy extension where strategies states can change the context's state. The Context changes its behavior because its own internal state changes. The state objects can trigger transitions between each other. The Context class stays unaware of what state comes next. Here the Context itself makes the choice.

My problem calls for Strategy rather than State because the Client chooses the operation, like to add or divide, while the Calculator never decides to change itself. The AddStrategy does not trigger to switch to DivideStrategy, and there is no internal state or automatic transition. Strategies here are just interchangeable tools.

Conclusion

The Strategy pattern gives the flexibility to Calculator by letting it call strategy.execute(a, b) and without knowing what operation it is doing. I can add new operation classes, and the existing code will not change, but only the Client will add a line to set the strategy.

Instead of having one Calculator class with if and switch statements, I have one Strategy interface, four concrete strategies, one Context class and one Client class. Here each concrete strategy has one job, which is easier to test, and adding a new concrete strategy will not break the existing code.